COMP4423 Assignment 2: Campus Vegetation Recognition

1 Introduction

This report presents the design, implementation, and analysis of an end-to-end **campus vegetation image classification pipeline** developed for this assignment. The core objective is to apply the standard machine learning paradigm—**data collection → dataset structuring/labeling → feature extraction → traditional classifier training → evaluation → deployment**—to a self-collected dataset containing eight plant categories captured in campus, under real-world variation (background, lighting, distance, occlusion, and viewpoint).

Unlike deep learning solutions (explicitly excluded), this project adopts a **traditional computer vision + classical machine learning** approach. In the provided codebase, the training workflow (train.py) builds reusable feature representations from images via dataset_loader.py, supporting multiple feature extractors such as **color histograms / color moments**, **HOG**, and texture features (e.g., **GLCM** and **LBP**), with local-descriptor features like **SIFT** (and **SURF** only when available). Candidate classifiers are trained and compared through a reproducible, seed-controlled pipeline, and the best-performing models are saved for later inference under ml_results/models/ (as referenced in README.md).

The implementation is organized around the delivery of the assignment:

- **Task 1–2 (Classes, collection, dataset building)**: data of 8 classes of vegetation are collected in campus, as shown in *Figure 1*.

Figure 1: Image examples for each class of vegetation in campus

Collected images or videos are organized into class folders and prepared through the existing dataset preparation pipeline described in following sections. Real-world variation is reflected by collecting the same class across different scenes and conditions, as well as applying strong data augmentation techniques.

- **Task 3–4 (Training, evaluation, error analysis)**: The pipeline in train.py trains multiple model–feature combinations, reports train/validation accuracy on the campus vegetation samples, and compute test accuracy for held-out test(which contains samples from internet purely for testing), and is designed to support standard evaluation artifacts (accuracy and per-class analysis), enabling discussion of failure modes attributable to hand-crafted features (e.g., lighting sensitivity for color features, background texture bias for texture descriptors, and viewpoint changes impacting local descriptors).
- **Task 5 (Application)**: A lightweight local application is implemented with **Gradio**, which loads saved models and allows users to **upload an image or select an existing sample** before returning a predicted plant class along with inference details (e.g. top probabilities/confidence if supported by the classifier).

The following sections detail the dataset design decisions, the feature engineering and model selection rationale, quantitative and qualitative evaluation results, and how the

final application operationalizes the trained pipeline for practical, local use.

2 System Design Methodology

2.1 Design philosophy: modular, reproducible pipeline

The codebase is organized around a **modular separation of concerns** to support maintainability and reuse. Throughout the codebase, a **seed=42** is used for random operations for **reproducibility**.

2.1.1 Data Preparation & Loading (Task 1-2)

The data pipeline is organized into three stages: **frame extraction** (raw videos -> images), **dataset expansion** (to increase samples and intra-class variation), and **supervised loading & feature extraction** (class folders -> labeled arrays of features for training).

Stage 1: Frame extraction

Frame extraction is handled by extract_grouped.py. Given a raw_grouped folder containing .MOV recordings, the script mirrors the directory structure into extracted_grouped and saves JPEG frames at a configurable interval (via the --interval-ms CLI argument). For example, an interval ms of 500 extracts 6 frames from a 3 seconds video. *Figure 2* shows the extracted images from the raw_grouped folder.

Class	Videos (raw)	Images (raw)	Extracted images
anthurium andraeanum	1	6	8
arachnothryx leucophylla	2	0	11
camellia japonica	3	0	13
codiaeum variegatum	2	3	14
ficus microcarpa	6	0	25
juniperus chinensis	3	0	13
livistona chinensis	4	0	14
rhododendron	7	1	29

Figure 2: Table showing the number of extracted images from raw videos and images (from /saved_logs/frame_extract_result.txt)

Stage 2: Dataset augmentation & expansion

Dataset expansion is needed due to the small number of samples collected. Dataset expansion is performed by expand_dataset_offline.py. This stage creates the final training dataset folder (by default extracted_group_augmented) by copying original images and writing augmented images to disk. Augmentations are designed to reflect real-world intra-class variation that can affect plant classification, including (but not limited to):

- geometric transformations (random resized crop, flips, rotation)
- illumination/photometric changes (contrast/brightness scaling, HSV Value-channel scaling, gamma correction)
- mild blur/noise corruption.

The script also balances sample size among classes, and uses a deterministic seed (--seed) to make the augmentation reproducible between runs. The result of existing dataset is shown in *Figure 3*.

Class	Original Images	Augmented images
anthurium andraeanum	8	200
arachnothryx leucophylla	11	200
camellia japonica	13	200
codiaeum variegatum	14	200
ficus microcarpa	25	200
juniperus chinensis	13	200
livistona chinensis	14	200
rhododendron	29	200

Figure 3: Table showing the number of original images and augmented images (from /saved_logs/data_augment_result.txt)

Stage 3: Supervised loading & Feature extraction with dimensionality reduction

Supervised loading is implemented in dataset_loader.py. The loader resolves the final dataset directory (typically extracted_group_augmented), scans each class folder, and assigns the folder name as the label while collecting all valid image files, then loads images with OpenCV and resizes them to a fixed img_size (default 128×128) for consistent feature extraction.

Feature extraction in this project is implemented in feature_extractor.py and uses traditional CV descriptors like color histogram, color moment, hog, glcm, lbp, sift, and optionally surf if available in the opencv library. Multiple descriptors can be concatenated into a single feature vector (e.g., color_hist+lbp+sift) for training different model–feature pipelines. To control dimensionality and prevent some high-dimensional blocks (e.g. HOG) dominating the classifier, the pipeline applies PCA on HOG via reduce_base_features_with_pca() and reduce its dimensionality to 400, then concatenates the reduced blocks for downstream training and inference. *Figure 4* shows the dimensionality of feature blocks and an example feature vector involving multiple features after applying PCA to each block.

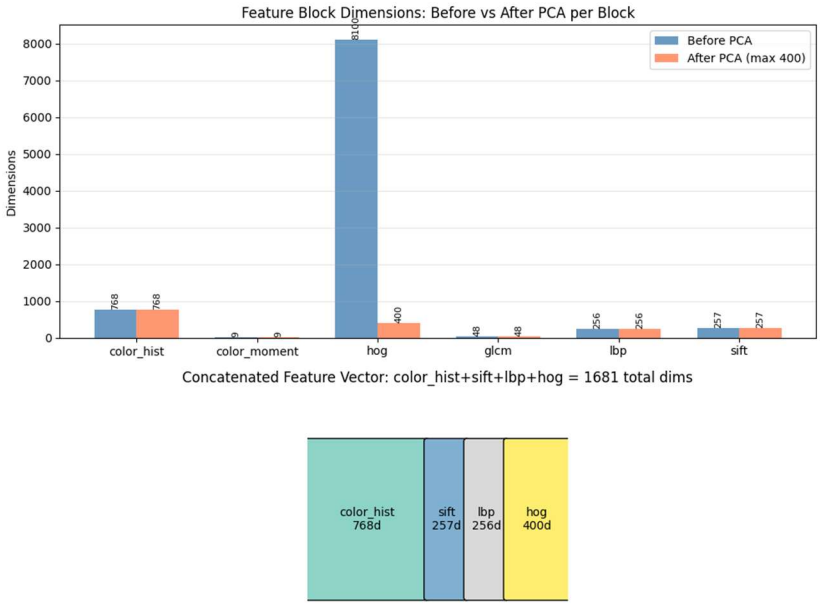


Figure 4: Feature block dimensions before and after PCA

2.1.2 Model Selection (Task 3-4)

Training is implemented in train.py with a reproducible configuration object (PipelineConfig). The three machine learning models used are **SVM**, **RandomForest** and **KNN**. The configurations for these models are handled in model.py.

SVM is implemented as an RBF-kernel SVC with class_weight="balanced"

and a fixed random_state=seed, providing a strong non-linear baseline for high-dimensional handcrafted features.

RandomForest is configured with n_estimators=400, n_jobs=-1, class_weight="balanced_subsample", and min_samples_leaf=5, making it robust to noisy features and class imbalance while remaining efficient on CPU.

KNN is implemented with n_neighbors=25 and weights="distance" (also using n_jobs=-1), providing a simple distance-based baseline that can perform well when classes form separable clusters in the handcrafted feature space.

2.1.3 Training Pipeline Design (Task 3-4)

For each (model, feature-combination) candidate, the following pipeline is carried out:

1. Features for each image are extracted in advance in the data loading step, with different methods (hog, lbp, sift etc.).
2. perform stratified cross-validation using StratifiedKFold, high dimension features will be reduced by PCA in each fold
3. compute average validation accuracy all folds.
4. Fit a final estimator on the full CV pool
5. Evaluate the estimator with the held-out test set to show its accuracy in real world scenarios
6. Save the trained models (currently only models with top 3 validation accuracy are saved) under ml_results/models/ for direct reuse by the local Gradio application.

2.1.4 Application Inference Pipeline (Task 5)

The saved models from the training pipeline can then be used to support the functionality of the standalone Gradio application in vegetation_classifier_app/app.py. *Figure 5* shows the interface of the application.

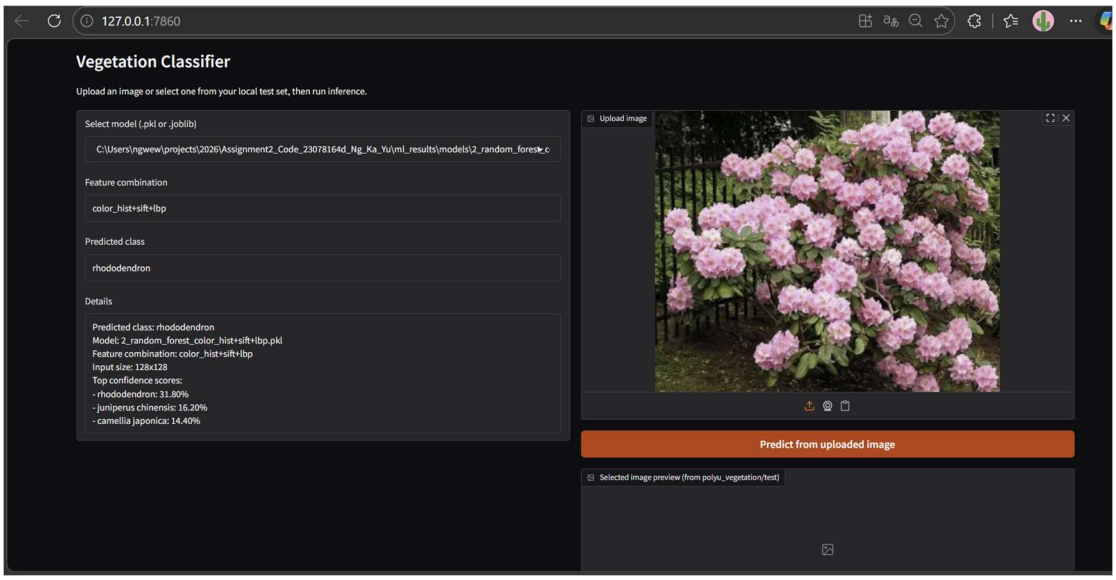


Figure 5: The panel on the left shows detail of the inference; The panel on the right allows user to ***upload custom images*** *or* ***select images from local directory*** *for inference.*

3 Testing & Robustness in real scenario

To ensure the robustness of the classifier application, a "progressive testing" strategy is used. Results from each run are selectively saved and evaluated. The following are some techniques used to test the application.

3.1 Cross validation

A 5-fold stratified cross validation is implemented during training to prevent overfitting and to have more accurate model performance evaluation. This is because a single train/val split can sometimes be "lucky" or "unlucky", having large difference in validation accuracy for a different split.

```
[random_forest | color_moment]
val_acc=0.7073, val_folds=[0.7812, 0.7031, 0.6562, 0.
6458, 0.7500], train_acc=1.0000, time=12.7s
```

Figure 6: train logs for a candidate (validation accuracies for folds range from 0.7812 to 0.6458)

Another reason that justifies cross validation implementation is that the collected dataset size is too small, so it would be more beneficial if the entire dataset can be used for training.

3.2 Per-combination feature analysis

feature combinations are progressively tested.

Initially, the project uses a one-feature-per-combination. However, models can only fit accurately (val_accuracy > 0.7) with color features (e.g. color histogram). Therefore, an approach to use a combination of a color feature and other feature is used. *Figure 7* lists the performance of top 3 feature combinations for each models, ordered by the held-out test accuracy. Generally, for each model, as validation accuracy increases, test accuracy increases. This represents that the models can effectively learn from the data and classify unseen images of vegetations.

(Note: Val Acc refers to the average validation accuracy for all folds during cross validation)

Model	Feature combination	Test Acc	Val Acc
RandomForest	color_hist, color_moment, sift, lbp, hog	79.07%	93.00%
RandomForest	color_hist, sift, lbp	79.07%	92.62%
RandomForest	color_hist, sift, lbp, hog	76.74%	92.88%
SVM	color_hist, sift, lbp	76.74%	92.13%
SVM	color_hist, color_moment, sift, lbp, hog	74.42%	91.63%
SVM	color_hist, sift, lbp, hog	74.42%	91.63%
KNN	color_hist, sift	60.47%	79.25%
KNN	color_hist, sift, lbp	55.81%	79.87%
KNN	color_hist, sift, glcm	55.81%	79.13%

Figure 7: Ranking top 3 feature combinations for each model by held-out test accuracy (based on 1600_samples_result.txt)

3.3 held-out test

The held-out test contains 43 samples outside of training set and is intended to evaluate the classifier's robustness in real world scenarios. The samples are

collected by downloading online images that match the vegetation's name. As some vegetations have vastly different looks due to differences in season, variant (e.g. different flower color, in bloom, etc.), only images that at least resemble the campus vegetation (i.e. no extreme intra-class variation that are not observable in the campus) are collected. For example, *Figure 8* is a sample of class "camellia japonica" collected in the campus, while *Figure 9* shows two images of camellia japonica in different season/variant.

Figure 8: a sample of camellia japonica collected in the campus

Figure 9: two images of camellia japonica collected online

3.4 Confusion matrix

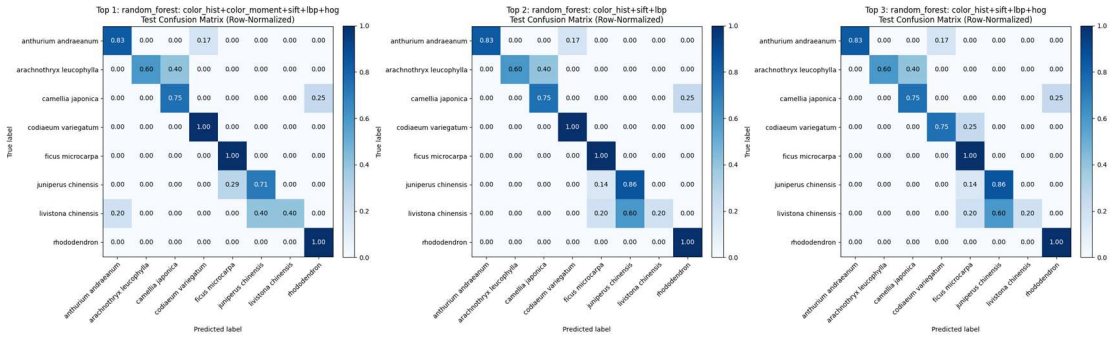
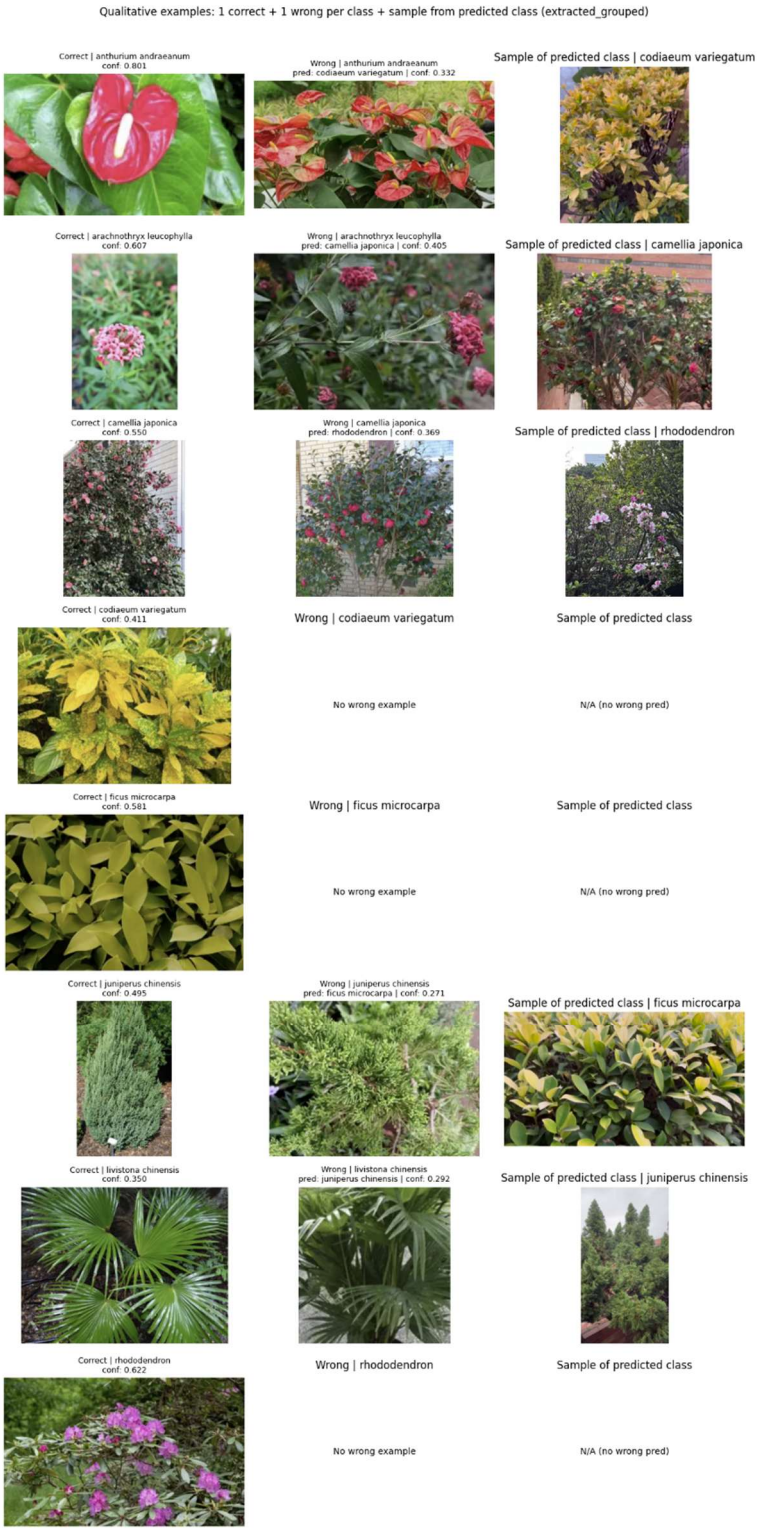


Figure 10: confusion matrices for top 3 candidate (model and feature combination)

The confusion matrix for the top 3 candidates shows that models trained can classify most classes correctly and confidently. However, all 3 candidates fail to achieve high accuracy (<70%) for two classes, namely “arachnothryx leucophylla” and “livistona chinensis”.

3.5 Qualitative Examples and failure modes



For “anthurium andraeanum”, the only wrongly predicted label is another vegetation that doesn’t resemble the vegetation of the true class. It is likely the

intra-class variation for anthurium andraeanum here is not recognized by the classifiers, likely due to the lack of stronger data variability for “anthurium andraeanum”.

For “arachnothryx leucophylla”, 40% of the is misclassified as “camellia japonica”. This is likely due to the scale of the test sample, which does not clearly show the shape of the blooms of “arachnothryx leucophylla”, together with the bushes in the background, making it resemble camellia japonica and led to the false positive. The same kind of false positives appears to be present for “camellia japonica” class and “rhododendron” class too.

For “juniperus chinesis” and “livistona chinesis”, the wrong predictions are likely the result of unsatisfactory shape feature extraction, as the vegetations are classified as another class of similar color, meaning that features like shape or texture are not being taken into account properly.

3 Challenges & Solutions & Reflections

3.1 Sample size concerns

First, the approach of extracting frames from videos is adopted. Neighboring frames from the same clip will look alike if the extraction interval is too short. Also, I realized the speed of angle changes in the videos are too slow, such that I had to adopt a 1000ms interval. However, this greatly reduces the sample size. As I was unavailable to obtain more data before the deadline of the assignment, my solution is to apply data augmentation to artificially create more training data.

3.2 Data quality

Due to the lack of samples, some distinct features or variation in each class may not be captured sufficiently. The solution I adopted is to apply more “extreme” data augmentation, to create more variations like scale, illumination, occlusions, color, noise etc. However, its effectiveness is limited and it may also create data that contributes negatively to training, and require manual filtering.

3.3 Feature dominance

During initial training, feature combinations that involve the usage of HOG

struggles to fit the data. It is then found out that HOG's high dimensionality (~8000) likely dominates the feature and disrupts the learning of the classifiers. Therefore, dimensionality reduction is done on HOG feature by PCA.

In addition, in hopes of improving accuracy, I tested out approaches to reduce dimensionality on other high dimension features, like color histogram. However, A substantial decrease in accuracy is observed. Eventually, since color features are crucial to identify vegetations, I concluded that some features can have high dimensionality, and letting it "dominate" the classification may not always be bad.

4 GenAI Usage & Reflection

4.1 How GenAI assisted implementation

GenAI was used to assist development of most modules (e.g. data loading, feature extraction, training/evaluation, and the Gradio app). It acted as an implementation partner: I specified what each part should do, how it should fit the assignment (traditional ML, fixed dataset layout under polyu_vegetation/), and which constraints mattered (reproducibility, no deep learning for Tasks 3–4, integration with existing scripts). I steered the process by breaking work into steps, reviewing generated code for correctness and fit with our data pipeline, and iterating when behaviour or interfaces needed to change. Final design choices, experiment decisions, and responsibility for the submitted work remain mine.

5 Conclusion

In summary, handcrafted features with ensemble and margin-based classifiers delivered solid results on augmented training data, while the small, possibly out-of-distribution test set remains the main caveat for claiming robust generalization.